

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

3 créditos



Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: Borys Alexy Panezo Guerrero

PARALELO: A

SEMESTRE: Quinto

ACTIVIDAD: # 8 — Resolución de Ejercicios

PERIODO ABRIL - AGOSTO 2026





Actividad # 8: Desarrollo de una API REST para Help Desk

Objetivo

Construir la arquitectura backend y la base de datos de un Sistema de Gestión de Incidentes, exponiendo una API RESTful segura que permita crear, consultar, actualizar y eliminar tickets.

Tecnologías utilizadas

- Java 21 y Spring Boot 3.5.16 para el servicio web.
- Spring Data JPA para persistencia y separación entre controlador, servicio y repositorio.
- PostgreSQL 17 como base de datos relacional.
- Flyway para crear y versionar la estructura de la tabla tickets.
- Spring Security con autenticación HTTP Basic y sesiones sin estado.
- JUnit, MockMvc y H2 para las pruebas automáticas.
- Docker para ejecutar la API y PostgreSQL en un entorno aislado.

Arquitectura y estructura del proyecto

La solución utiliza una arquitectura por capas. El controlador recibe las peticiones HTTP; el servicio contiene la lógica de negocio y las transacciones; el repositorio gestiona la persistencia; y PostgreSQL conserva los datos. Las credenciales se reciben mediante variables de entorno y no forman parte del código fuente.

Componente	Responsabilidad
TicketController	Expone los cinco endpoints REST.
TicketService	Aplica reglas, transacciones y tratamiento de datos.
TicketRepository	Ejecuta las operaciones de persistencia JPA.
PostgreSQL	Almacena tickets e índices de consulta.
ApiExceptionHandler	Devuelve errores JSON uniformes.
SecurityConfig	Protege todas las rutas mediante HTTP Basic.

Diseño de la base de datos

La migración V1 crea la tabla tickets con los atributos mínimos solicitados: id, título, descripción, categoría, prioridad y estado. También registra fechas de creación y actualización e índices para estado y prioridad.

```
CREATE TABLE tickets (  
  id BIGSERIAL PRIMARY KEY,  
  titulo VARCHAR(150) NOT NULL,  
  descripcion VARCHAR(2000) NOT NULL,  
  categoria VARCHAR(20) NOT NULL,  
  prioridad VARCHAR(20) NOT NULL,  
  estado VARCHAR(20) NOT NULL DEFAULT 'ABIERTO',  
  fecha_creacion TIMESTAMP NOT NULL,  
  fecha_actualizacion TIMESTAMP NOT NULL,  
  INDEX (estado),  
  INDEX (prioridad);
```



```
    creado_en TIMESTAMP WITH TIME ZONE NOT NULL,  
    actualizado_en TIMESTAMP WITH TIME ZONE NOT NULL  
);
```

Valores admitidos: categoría RED, HARDWARE o SOFTWARE; prioridad ALTA, MEDIA o BAJA; estado ABIERTO, EN_PROGRESO o CERRADO.



Implementación de la API REST

Método	Ruta	Resultado	Código
GET	/tickets	Lista todos los tickets	200
GET	/tickets/{id}	Busca un ticket	200/404
POST	/tickets	Registra un ticket	201/400
PUT	/tickets/{id}	Actualiza un ticket	200/400/404
DELETE	/tickets/{id}	Elimina un ticket	204/404

Código principal del controlador

```
package ec.edu.utm.helpdesk.ticket;

import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.net.URI;
import java.util.List;

@RestController
@RequestMapping("/tickets")
public class TicketController {
    private final TicketService service;

    public TicketController(TicketService service) { this.service = service; }

    @GetMapping public List<Ticket> listar() { return service.listar(); }
    @GetMapping("/{id}") public Ticket buscar(@PathVariable Long id) { return
service.buscar(id); }

    @PostMapping
    public ResponseEntity<Ticket> crear(@Valid @RequestBody TicketRequest request) {
        Ticket creado = service.crear(request);
        return ResponseEntity.created(URI.create("/tickets/" +
creado.getId())).body(creado);
    }

    @PutMapping("/{id}")
    public Ticket actualizar(@PathVariable Long id, @Valid @RequestBody
TicketRequest request) {
        return service.actualizar(id, request);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> eliminar(@PathVariable Long id) {
        service.eliminar(id);
        return ResponseEntity.noContent().build();
    }
}
```



Pruebas y resultados

Las pruebas se ejecutaron contra PostgreSQL 17 en contenedores aislados. La API realizó correctamente el ciclo CRUD completo. Adicionalmente se ejecutaron pruebas automáticas con el siguiente resultado: 3 pruebas, 0 fallos, 0 errores y 0 omitidas.

Help Desk API — Prueba con cURL

HTTP 201 Created

PETICIÓN AUTENTICADA

POSThttp://localhost:8080/tickets

CUERPO JSON ENVIADO

```
{ "titulo": "Falla de red en laboratorio", "descripcion": "Los equipos no tienen acceso a Internet", "categoria": "RED", "prioridad": "ALTA", "estado": "ABIERTO" }
```

RESPUESTA REAL DE LA API

```
{  "id": 1,  "titulo": "Falla de red en laboratorio",  "descripcion": "Los equipos no tienen acceso a Internet",  "categoria": "RED",  "prioridad": "ALTA",  "estado": "ABIERTO",  "creadoEn": "2026-08-06T06:08:05.652929134Z",  "actualizadoEn": "2026-08-06T06:08:05.652929134Z" }
```

Actividad 8 · Spring Boot · PostgreSQL · Respuesta capturada automáticamente el 6 de agosto de 2026

Figura 1. Creación de un ticket — HTTP 201 Created.



Help Desk API — Prueba con cURL

HTTP 200 OK

PETICIÓN AUTENTICADA

GET

http://localhost:8080/tickets

RESPUESTA REAL DE LA API

```
[
  {
    "id": 1,
    "titulo": "Falla de red en laboratorio",
    "descripcion": "Los equipos no tienen acceso a Internet",
    "categoria": "RED",
    "prioridad": "ALTA",
    "estado": "ABIERTO",
    "creadoEn": "2026-08-06T06:08:05.652929Z",
    "actualizadoEn": "2026-08-06T06:08:05.652929Z"
  }
]
```

Actividad 8 - Spring Boot - PostgreSQL - Respuesta capturada automáticamente el 6 de agosto de 2026

Figura 2. Listado de tickets — HTTP 200 OK.



Help Desk API — Prueba con cURL

HTTP 200 OK

PETICIÓN AUTENTICADA

GET

http://localhost:8080/tickets/1

RESPUESTA REAL DE LA API

```
{
  "id": 1,
  "titulo": "Falla de red en laboratorio",
  "descripcion": "Los equipos no tienen acceso a Internet",
  "categoria": "RED",
  "prioridad": "ALTA",
  "estado": "ABIERTO",
  "creadoEn": "2026-08-06T06:08:05.652929Z",
  "actualizadoEn": "2026-08-06T06:08:05.652929Z"
}
```

Actividad 8 · Spring Boot · PostgreSQL · Respuesta capturada automáticamente el 6 de agosto de 2026

Figura 3. Consulta de un ticket por identificador — HTTP 200 OK.



Help Desk API — Prueba con cURL

HTTP 200 OK

PETICIÓN AUTENTICADA

PUT

http://localhost:8080/tickets/1

CUERPO JSON ENVIADO

```
{ "titulo": "Falla de red en laboratorio", "descripcion": "Conectividad restablecida y verificada", "categoria": "RED", "prioridad": "MEDIA", "estado": "CERRADO" }
```

RESPUESTA REAL DE LA API

```
{  "id": 1,  "titulo": "Falla de red en laboratorio",  "descripcion": "Conectividad restablecida y verificada",  "categoria": "RED",  "prioridad": "MEDIA",  "estado": "CERRADO",  "creadoEn": "2026-08-06T06:08:05.652929Z",  "actualizadoEn": "2026-08-06T06:08:05.889254225Z" }
```

Actividad 8 · Spring Boot · PostgreSQL · Respuesta capturada automáticamente el 6 de agosto de 2026

Figura 4. Actualización del estado del ticket — HTTP 200 OK.

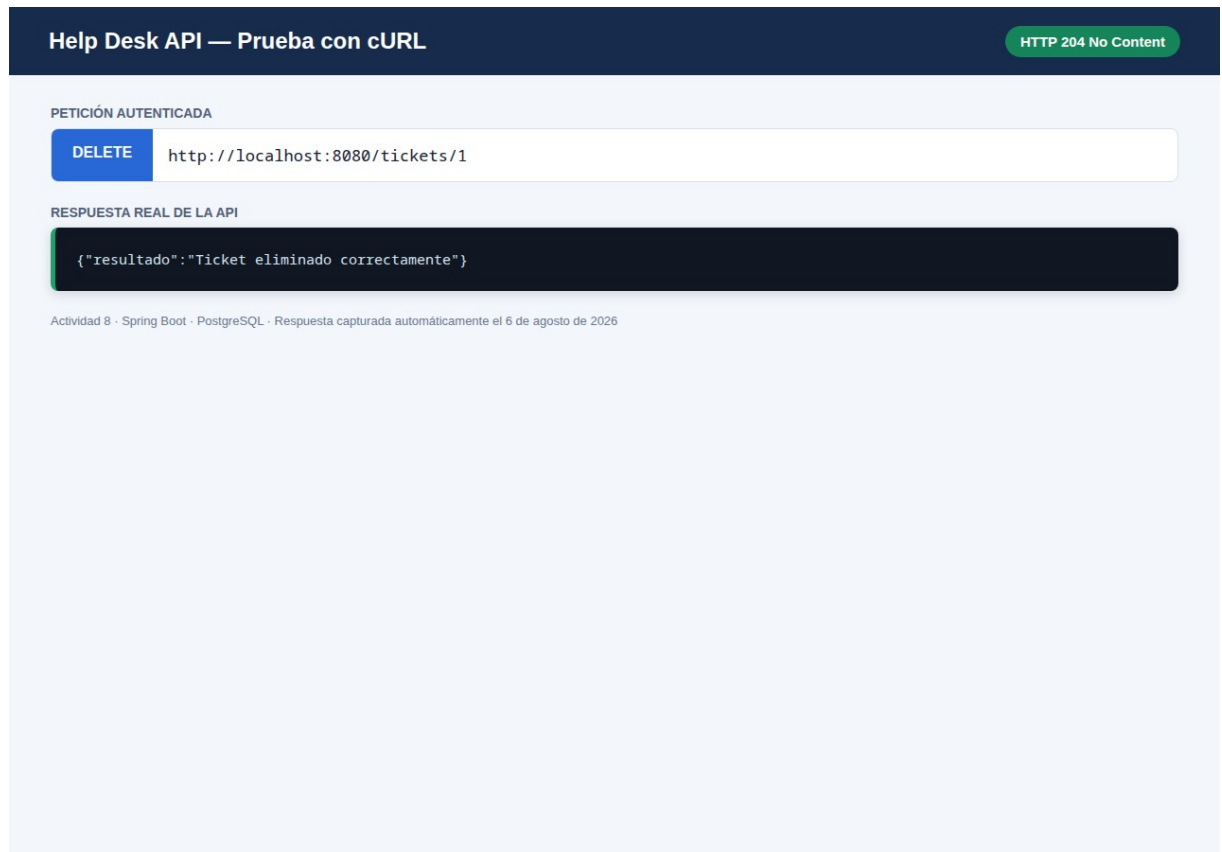


Figura 5. Eliminación del ticket — HTTP 204 No Content.

Conclusiones

La API desarrollada cumple los requisitos funcionales de la Actividad 8: utiliza PostgreSQL, implementa la estructura mínima de tickets, expone las cinco operaciones CRUD y devuelve información en JSON. La validación de entradas, los errores estandarizados, las migraciones y la autenticación mejoran la seguridad y mantenibilidad del servicio.

El proyecto puede ejecutarse localmente mediante Docker y contiene una colección Postman, pruebas automáticas y documentación de uso. El código se organizó en la rama feature/backend-api para integrarlo posteriormente en develop, siguiendo el flujo de control de versiones solicitado.

Repositorio

Código fuente: <https://github.com/JBorys687/helpdesk-datacenter>

Bibliografía

- Spring. (2026). Spring Boot Reference Documentation. <https://docs.spring.io/spring-boot/>
- Spring. (2026). Spring Data JPA Reference Documentation. <https://docs.spring.io/spring-data/jpa/reference/>



- PostgreSQL Global Development Group. (2026). PostgreSQL Documentation.
<https://www.postgresql.org/docs/>
- Fielding, R., Nottingham, M., & Reschke, J. (2022). HTTP Semantics (RFC 9110).
<https://www.rfc-editor.org/rfc/rfc9110>